

CSCI 4160/6963, ECSE 4965/6965

Reinforcement Learning

Homework 4

Overview

In this homework, you will use the upper confidence bound (UCB) algorithm to identify a good controller for a benchmark RL problem. You will work with the Mountain Car (MC) benchmark implemented in the OpenAI Gymnasium package. In MC, you need to learn a controller to drive a car up a hill. Since the car doesn't have enough power, you need to back up the left hill first in order to gain speed. After each step, you get a small negative reward (smaller if you apply larger thrust); when you get to the goal, you get a positive reward of 100. MC is an interesting benchmark because it's quite easy to solve for a human, yet it is hard for RL because it gives sparse rewards (you only get a positive reward when you reach the goal).

The MC simulator has a continuous state space (position is between $[-1.2, 0.6]$, velocity is between $[-0.07, 0.07]$, and thrust is in $[-1, 1]$, where negative thrust means "left" and positive thrust means "right"). The starting position is chosen randomly in $[-0.6, -0.4]$, where the bottom of the valley is at around -0.5 ($-\pi/6$, to be precise) and starting velocity is 0.

Your job will be to use UCB to find a simple controller that achieves a high reward. You will first try to find a simple policy that gets you to a reward of 95. After that, you will discretize the state and action space and use UCB to find a controller that achieves a reward of 98 (**98.5 for graduate students**).

Logistics

All coding assignments in this course will be in Python. If you need help with Python, please talk to me ahead of time, so we can discuss the best way to get familiar with it.

For this assignment, you will be using your own computer, which should be sufficient. The code should not require more than 5 minutes to run on any standard laptop.

You are provided with skeleton starting code. Please use the Python libraries in the `requirements.txt` file that's provided on LMS.

OpenAI environments are nice because they are all run in the same way, with very little changes from one to the next. To create the environment, you need to run

```
env = gym.make('MountainCarContinuous-v0').env.unwrapped
```

where the final unwrapped argument just makes sure you can modify the environment state directly (which you will need to do in a future homework).

If you want to render the environment, you also need to set the `render_mode='human'` flag:

```
env = gym.make('MountainCarContinuous-v0',  
render_mode='human').env.unwrapped
```

Once you have an environment, you will need a control policy, π , that gives you an action for each state. Given your action, you can call the `step` function of the environment:

```
observation, reward, done, __, __ = env.step(action)
```

where `observation` is the next state, `reward` is the one-step reward, and the boolean `done` is set to true when you reach the goal. You shouldn't need the other two variables returned by `step`. So, once you call `step`, you can use `observation` to decide your action in the next step.

In the case of MC, the one-step reward is calculated as follows:

```
reward := -0.1 * action2
```

You get a reward of 100 when you reach the goal. Thus, you are penalized for larger thrusts. In order to maximize the reward, you need to use small-magnitude thrust as much as you can.

Don't forget to call `env.reset()` every time you start a new episode!

Grading

The main part of this assignment is to set up the MC control problem as a bandit problem so that you can apply UCB. At a high level, your task will be to design a number of different control policies – each policy will be an “arm” that will return a random reward every time you “pull it”, i.e., every time you run that controller for a full episode. In particular, to get a full score, you need to implement the following two parts.

- 1) Implement a simple policy that achieves a reward of at least 95 (as averaged over 100 episodes). This policy can be a simple set of rules (e.g., if the velocity is in some range and the position is in some range, then apply some predefined thrust).
- 2) Design a number of different controllers and implement the UCB algorithm so as to find a controller that achieves a reward of 98 (**98.5 for graduate students**). To come up with a set of possible controllers, you can discretize the state and action space, similar to item 1 above (see the hints for some suggestions).

Note that the final controller you choose needs to be evaluated on 100 episodes (use UCB to find this controller and then run it for 100 episodes to calculate the reward).

In both parts, you will lose 5 points for each 1 point of reward below the target.

Make sure your code prints out some indication of progress as well as all required information – we should not have to insert any print statements in order to find out whether your code works!

Finally, please provide answers to each of the questions below:

- 1) Explain how you discretize the state-action space and why you think this approach makes sense when it comes to finding a good controller.
- 2) Why does using the UCB algorithm makes sense for this application? What guarantees does the UCB algorithm provide?

Hints and Tips

- I suggest your start by playing with the simulator a bit and maybe building a simple policy first. This will give you a feel for how the environment works and what the challenges are. Once you understand what good policies look like, it should be easier to set up the “arms” for the UCB algorithm, i.e., different policies to choose from.
- For a reward of 98.5, it is possible to find a good controller by discretizing the state space into 4 parts and the action space into 3, which gives you a total of 3^4 controllers to choose from.
- When you call the `step` function, make sure the action variable is in a list/array. The OpenAI simulator may crash otherwise.
- The `num_steps` variable is set to 1000, to allow for controllers that swing multiple times.
- To find a controller that achieves a reward above 98.5, you can run UCB and test if an arm has a lower bound of 98.5 or higher – once you find this controller, you can break out of the UCB loop and evaluate the controller over 100 episodes. This still has some probability of finding a bad controller, so think about how to address this issue.

Submission

Please use LMS to submit a zip file containing **only the following files**: 1) your **.py** code, along with instructions on how to run it, 2) a **.pdf** file containing your answers to the above questions. The deadline is **11:59pm, Thursday, Oct. 9**.